Actividad 3: Enfoque modular y diseño Top-Down

Proyecto CALCULOAREA

Lucas Del Pino, Federico Mattia, Maximo Orue y Ulises Ferrari

EEST N.º 1 “Eduardo Ader”, Vicente López

Laboratorio de Programación, 5.º 3.º A-B

Prof. Mansilla Muñoz York Elías

17 de septiembre de 2026

Resumen

El presente informe documenta el desarrollo de la Actividad 3 de Laboratorio de Programación, centrada en el enfoque estructurado Top-Down y en la modularización mediante funciones en C++. El proyecto CALCULOAREA recibe un radio positivo, calcula el área de un círculo mediante la función calcularAreaCirculo y representa una diana en la consola de acuerdo con el tamaño del radio. La solución incorpora un prototipo de función, validación de entrada, una constante inmutable para PI y un valor de retorno. Asimismo, se documentan la estructura del repositorio, el proceso de compilación, las pruebas realizadas y la distribución de tareas del equipo.

Palabras clave: C++, modularización, Top-Down, función, validación, cálculo de área

Actividad 3: Enfoque modular y diseño Top-Down

El diseño Top-Down propone comenzar con el problema general y dividirlo progresivamente en partes más pequeñas, especializadas y fáciles de resolver. En C++, esta estrategia se puede implementar mediante funciones diseñadas por el programador. De acuerdo con la consigna, la modularización facilita la depuración, favorece la reutilización del código y permite organizar mejor el trabajo (EEST N.º 1 “Eduardo Ader”, 2026).

En esta actividad se aplica el enfoque al desarrollo de un programa denominado CALCULOAREA. El sistema solicita el radio de un círculo, verifica que el valor sea positivo, envía el dato a una función modular que realiza el cálculo y, finalmente, muestra el resultado y una representación ASCII de una diana. El código fuente completo del proyecto se encuentra disponible en el repositorio del equipo (Del Pino et al., 2026).

Objetivos

- Aplicar el enfoque de diseño Top-Down a un problema de programación sencillo.
- Separar el cálculo matemático en una función modular.
- Utilizar un prototipo de función antes de main().
- Validar el radio para impedir valores iguales o menores que cero.
- Utilizar una constante const para representar el valor de PI.

- Documentar y organizar el proyecto según la estructura de carpetas indicada en la actividad (EEST N.º 1 “Eduardo Ader”, 2026).

Integrantes y funciones

La consigna solicita que, cuando el informe sea grupal, se detalle el nombre de cada integrante y la tarea exacta realizada por cada uno (EEST N.º 1 “Eduardo Ader”, 2026). La distribución del trabajo del equipo se presenta en la Tabla 1.

Distribución de tareas del equipo

Integrante	Función o tarea
Lucas Del Pino	Desarrollo del algoritmo y de la función calcularAreaCirculo
Federico Mattia	Validación de datos y pruebas de compilación y ejecución
Maximo Orue	Diseño del diagrama de flujo y organización del enfoque Top-Down
Ulises Ferrari	Documentación del proyecto, README y elaboración del informe en formato APA 7

Diseño Top-Down y modularización

El programa se divide en responsabilidades claras. La función main() concentra la interacción con el usuario, la validación del dato y la presentación del resultado; por otro lado, calcularAreaCirculo encapsula la operación matemática. Esta separación hace que cada módulo tenga una función concreta y permite modificar o probar el cálculo sin mezclarlo con la entrada y salida de datos.

El flujo general es: ingreso del radio, validación, llamada a calcularAreaCirculo, devolución del área, presentación del resultado y representación ASCII de la diana. Si el radio no es válido, el flujo se detiene antes del cálculo.

Desarrollo del programa

Prototipo de función

El prototipo declara la existencia de la función antes de main(). Indica que calcularAreaCirculo recibe un valor double y devuelve otro valor double. Esto permite que el compilador conozca la firma de la función cuando encuentra la llamada dentro de main().

```
double calcularAreaCirculo(double radio);
```

Entrada y validación

El programa almacena el radio ingresado en radioEstudiante. Luego verifica la condición radioEstudiante <= 0. Cuando se cumple, muestra un mensaje de error y finaliza mediante return 1. De esta manera, no se realiza el cálculo con un dato geométricamente inválido.

```
cout << "=> Ingrese el radio del circulo/diana (en cm): ";
cin >> radioEstudiante;

if (radioEstudiante <= 0) {
    cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
    return 1;
}
```

Invocación de la función

Una vez validado el radio, main() invoca la función y guarda el resultado en la variable decimal areaFinal, tal como solicita la actividad (EEST N.º 1 “Eduardo Ader”, 2026).

```
double areaFinal = calcularAreaCirculo(radioEstudiante);
```

Cálculo modular del área

La función utiliza la fórmula geométrica $A = \pi \times r^2$. El valor de PI se declara como const para impedir modificaciones accidentales. Luego se calcula el área en una variable local y se devuelve mediante return.

```
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;
    double area = PI * r * r;
    return area;
}
```

Salida y diana ASCII

Después del cálculo, el programa muestra areaFinal expresada en centímetros cuadrados. Como parte del bonus de la actividad, utiliza estructuras if y else if para seleccionar una representación ASCII diferente según el rango del radio ingresado.

```
cout << "=> El area del circulo es: " << areaFinal << " cm^2" << endl;
```

Estructura del proyecto

La consigna exige una carpeta raíz llamada CALCULOAREA con archivos de configuración, documentación, código fuente y capturas (EEST N.º 1 “Eduardo Ader”, 2026). La estructura utilizada, que puede consultarse en el repositorio del equipo (Del Pino et al., 2026), es la siguiente:

```
CALCULOAREA/
├── .gitignore
├── LICENSE
├── README.md
├── Docs/
│   └── InformeProyectoCALCULOAREA.docx
├── src/
```

```
│   └── main.cpp
└── Capturas/
    ├── compilacion_area.png
    └── diagrama_flujo.png
```

Compilación y pruebas

La actividad indica compilar desde la raíz de CALCULOAREA mediante g++ y generar el ejecutable dentro de la carpeta src (EEST N.º 1 “Eduardo Ader”, 2026). Los comandos utilizados fueron los siguientes:

```
g++ src/main.cpp -o src/area.exe
.\src\area.exe
```

Para comprobar el comportamiento del programa se utilizó un radio positivo de 5 cm. El cálculo produjo aproximadamente 78,5398 cm² y el programa mostró la diana correspondiente a la categoría de radio pequeño. También se verificó el control de consistencia ingresando 0, ante lo cual el sistema mostró el mensaje de error y terminó sin efectuar el cálculo. La evidencia se presenta en la Figura 1.

Figura 1

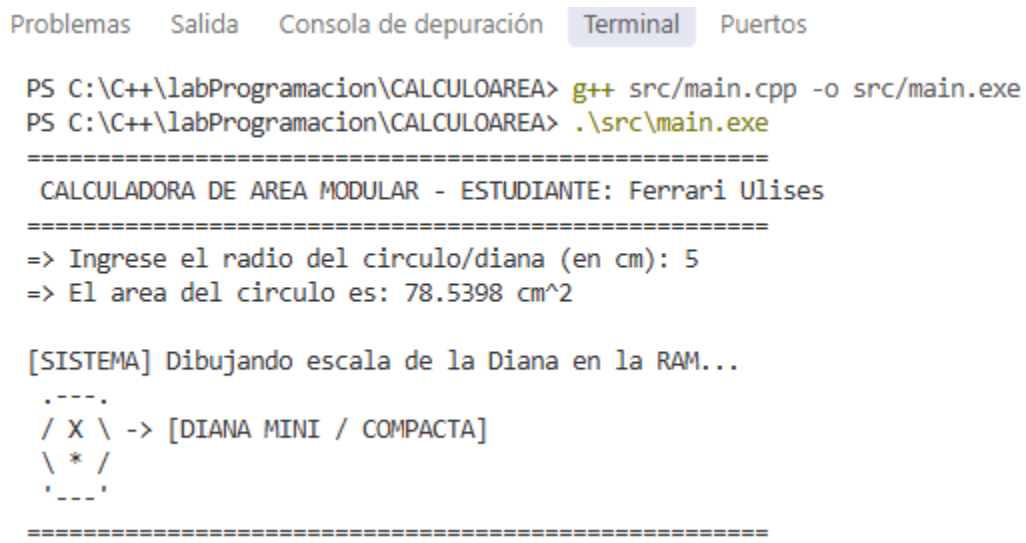
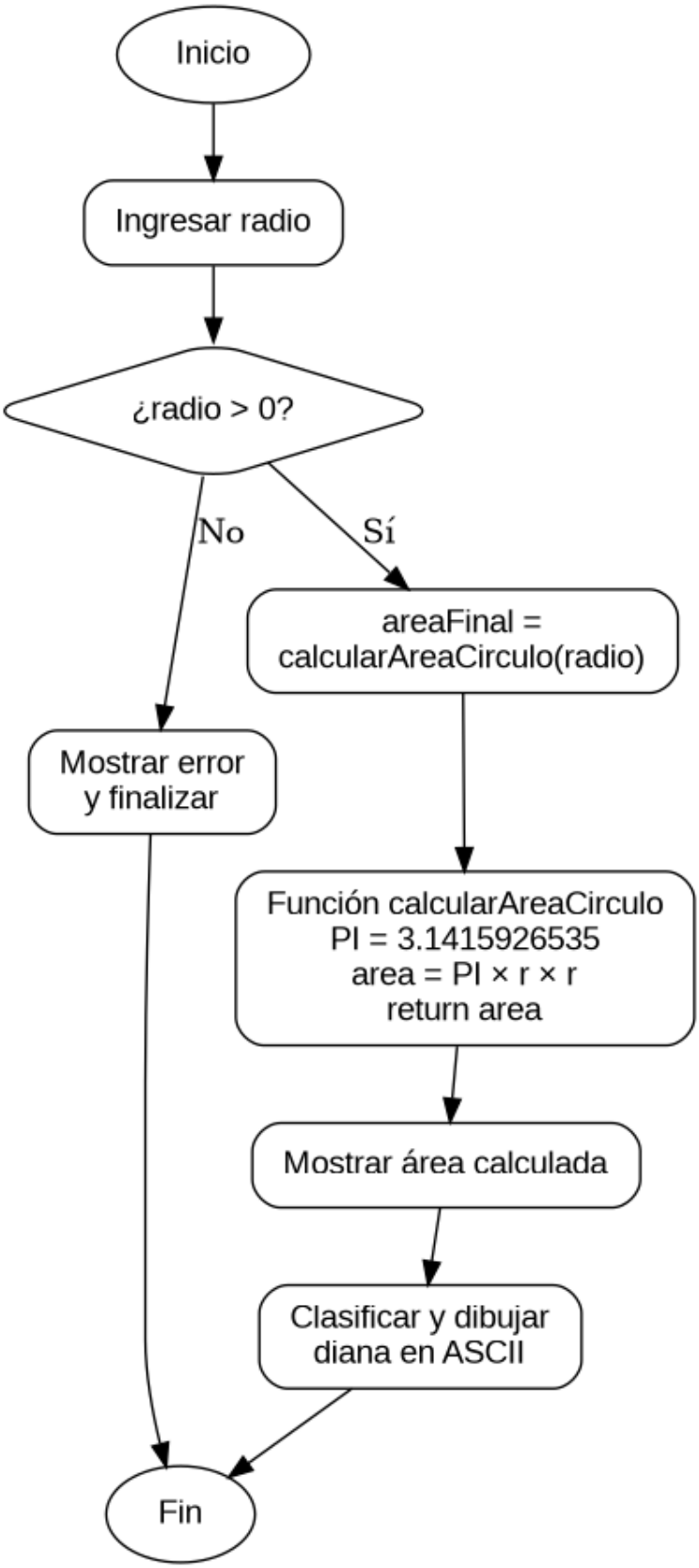


Diagrama de flujo

El diagrama representa el orden lógico de las operaciones: inicio, ingreso del radio, validación, cálculo modular del área, presentación del resultado y selección de la diana ASCII. La decisión de validación permite observar con claridad el enfoque Top-Down utilizado en la solución (ver Figura 2).

Figura 2


Inicio
Ingresar radio
¿radio > 0?
No
Sí
areaFinal =
calcularAreaCirculo(radio)
Mostrar error
y finalizar
Función calcularAreaCirculo
PI = 3.1415926535
area = PI × r × r
return area
Mostrar área calculada
Clasificar y dibujar
diana en ASCII
Fin

Cuestionario técnico

Pregunta 1. El prototipo de la función

Si se elimina el prototipo double calcularAreaCirculo(double radio);, el compilador llega a la llamada calcularAreaCirculo(radioEstudiante) dentro de main() sin haber recibido previamente la declaración de esa función. Como la definición completa aparece más abajo, en ese punto el compilador todavía no conoce su nombre, sus parámetros ni el tipo de dato que devuelve. Por eso puede producirse un error de compilación. El prototipo resuelve este problema al anunciar la firma de la función antes de main().

Pregunta 2. El filtro de consistencia

Un radio de 0 cm no describe un círculo con superficie positiva: el resultado del área sería 0. Un radio de -5 cm no representa una longitud física válida, porque una longitud no puede ser negativa. Por este motivo, el programa controla el dato antes de efectuar la operación y evita resultados basados en una entrada inválida.

Pregunta 3. Protección de constantes con const

Declarar PI como const impide que su valor sea modificado accidentalmente durante la ejecución. En este programa, mantener fijo el valor utilizado para el cálculo evita que una asignación incorrecta altere el resultado. En sistemas donde los datos deben permanecer estables, reducir las posibilidades de modificaciones no deseadas ayuda a prevenir errores de programación.

Conclusión

El proyecto CALCULOAREA cumple con el objetivo central de la actividad al aplicar un enfoque Top-Down y una organización modular en C++. La función calcularAreaCirculo concentra el cálculo del área, mientras que main() controla la interacción, la validación y la salida. La utilización de un prototipo, una constante const, una variable local y return permite aplicar los conceptos técnicos solicitados en la consigna (EEST N.º 1 “Eduardo Ader”, 2026). Las pruebas realizadas confirman que el programa calcula correctamente el área para un radio válido y rechaza entradas menores o iguales a cero.

La documentación del proyecto queda acompañada por su estructura de carpetas, el código fuente, el README, las capturas de evidencia y el diagrama de flujo, de manera que el trabajo pueda incorporarse al repositorio de GitHub exigido por la actividad (Del Pino et al., 2026).

Referencias

EEST N.º 1 “Eduardo Ader”. (2026). *Actividad 3: Enfoque modular y diseño Top-Down* [Consigna de la actividad, Laboratorio de Programación, 5.º 3.º A-B].

Ferrari, U. (2026). *Repositorio oficial del proyecto CALCULOAREA* [Repositorio de código fuente en GitHub]. GitHub. https://github.com/Ferraribip/calculoarea.git